

3. Beadandó feladat dokumentáció

Készítette: Wang Zhibo

Hallgató Harald E-mail:

crgyrx@inf.elte.hu

Feladat:

Készítsünk programot, amellyel a következő két személyes játékot játszhatjuk. A malom játékban két játékos egy 24 mezőből álló speciális játéktáblán játszhatja, ahol a mezők három egymásba helyezett négyzetben helyezkednek (mindegyikben 8, a sarkoknál és a felezőpontoknál), melyek a felezőpontok mentén össze vannak kötve. Kezdetben a tábla üres, és felváltva helyezhetik el rajta bábuikat az üres mezőkre. Az elhelyezés után a játékosok felváltva mozgathatják bábuikat a szomszédos (összekötött) mezőkre. Amennyiben egy játékos nem tud mozgatni, akkor passzolhat a másik játékosnak. Ha valakinek sikerül 3 egymás melletti mezőt elfoglalnia (azaz malmot alakít ki, rakodás, vagy mozgatás közben), akkor leveheti az ellenfél egy általa megjelölt bábuját (kivéve, ha az egy malom része). Az a játékos veszít, akinek először megy 3 alá a bábuk száma a mozgatási fázis alatt.

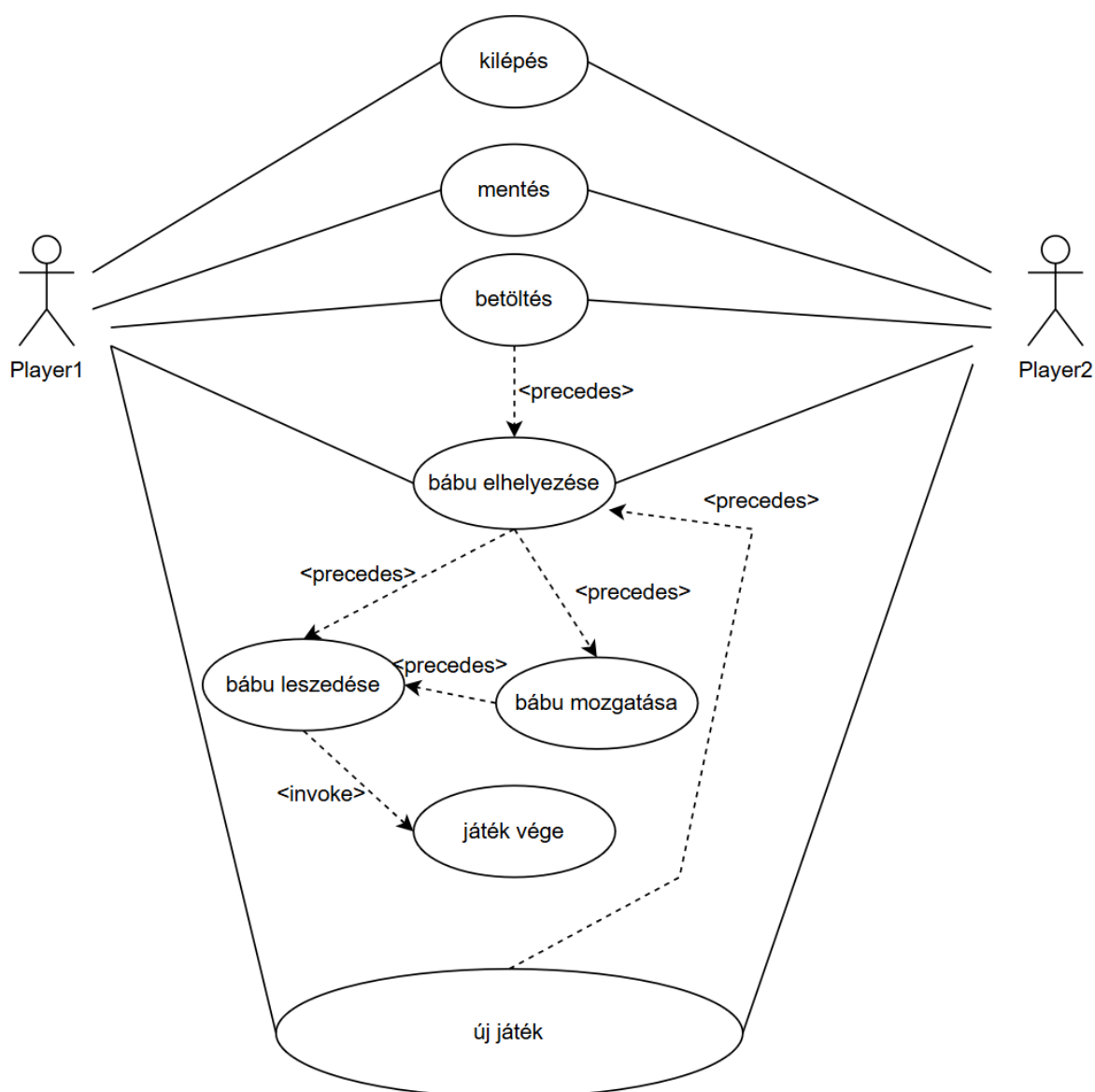
A program biztosítson lehetőséget új játék kezdésére, mentésre és betöltésre. Ismerje fel, ha vége a játéknak, és jelenítse meg, melyik játékos győzött.

Elemzés:

- A játékot két játékos játszhatja: Player1 (piros bábuk) és Player2 (kék bábuk). A program indításkor automatikusan új játékot indít, ahol a tábla üres, és Player1 kezd. A játék két fázisból áll: helyezési fázis (Placing), ahol 9-9 bábút helyeznek el felváltva, valamint mozgatási fázis (Moving), ahol a bábuikat szomszédos mezőkre mozgathatják. Nincs nehézségi szint, a játékosok lépései alapján dől el a győztes.
- A feladatot .NET Avalonia alkalmazásként valósítjuk meg, elsődlegesen asztali környezetre, de a szerkezetet úgy alakítjuk ki, hogy platformfüggetlen legyen. Az alkalmazás egy fő nézetből fog állni.
- A játék nézete tartalmazza a játéktáblát, a játék státuszát (aktuális játékos, fázis) a lap alján, valamint a menürendszert (Új játék, Mentés, Betöltés, Kilépés) az ablak tetején. Játék betöltésére és mentésére az adott operációs rendszer (pl. Windows, Linux, macOS) fájlkezelő dialógusablakain keresztül van lehetőség.
- A játéktáblát egy 24 nyomógombokból álló rács reprezentálja, amelyek a pálya kereszteződésein helyezkednek el. A nyomógomb egérekattintás

hatására végzi a lépést: helyezéskor bábút helyez, mozgatáskor kiválasztja és mozgatja a bábút, leszedéskor eltávolítja az ellenfél bábuját. A bábuk színe: piros (Player1), kék (Player2), fehér (üres). Malom kialakulásakor üzenetdoboz jelenik meg, és a leszedési mód aktiválódik.

- A játék automatikusan jelzi előugró üzenettel (MessageBox), ha malom alakult ki, ha hibás lépés történt, vagy ha vége a játéknak.
- A felhasználói esetek az 1. ábrán láthatóak.

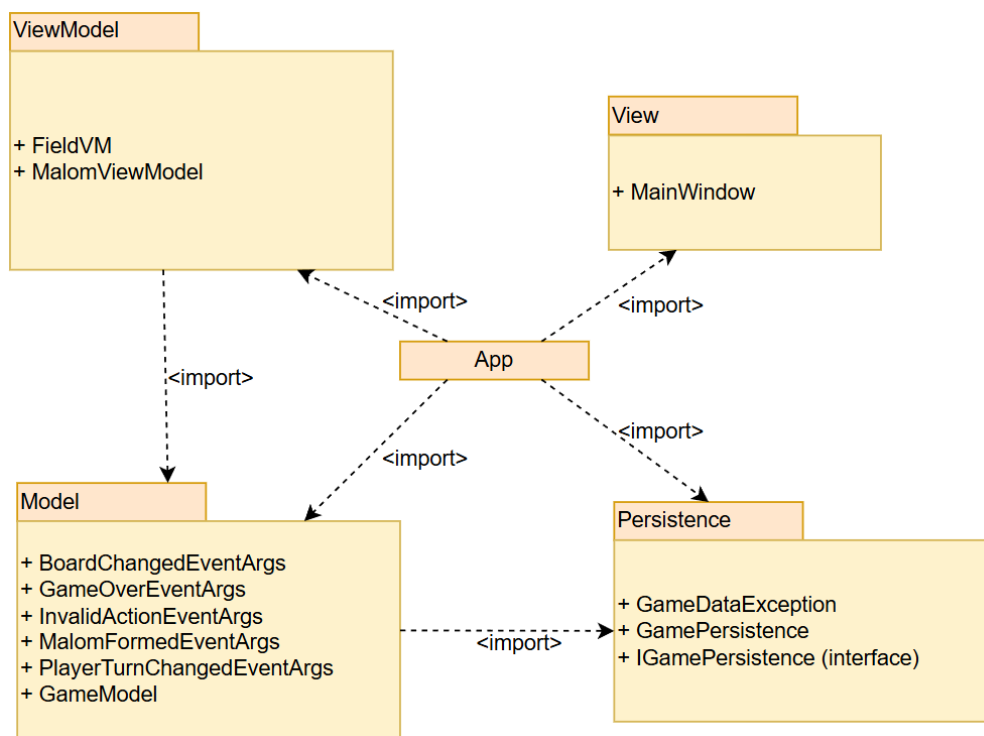


1. ábra: Felhasználói esetek diagramja

Tervezés:

Programszerkezet:

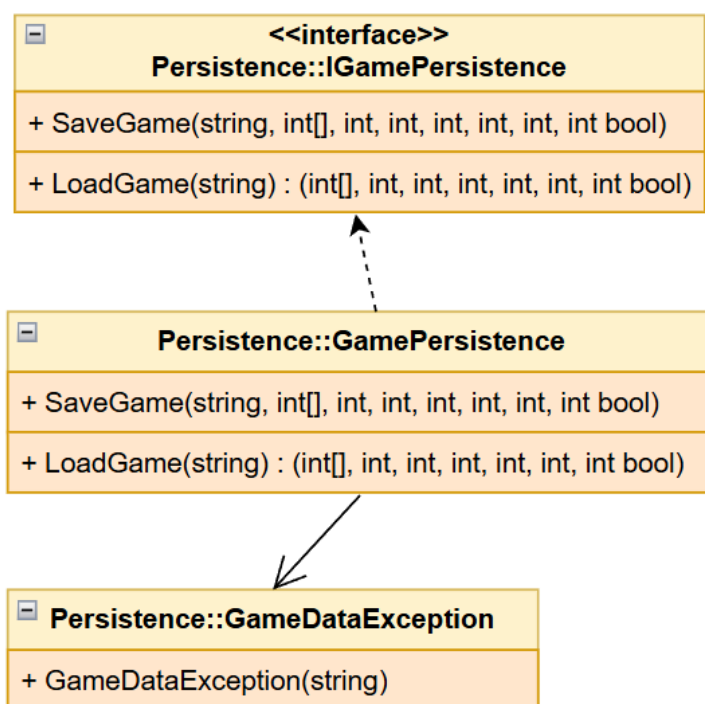
- A programot MVVM architektúrában valósítjuk meg, ennek megfelelően **View**, **Model**, **ViewModel** és **Persistence** névttereket valósítunk meg az alkalmazáson belül. A program környezetét az alkalmazás osztály (**App**) végzi, amely példányosítja a modellt, a nézetmodellt és a nézetet, biztosítja a kommunikációt, valamint felügyeli az adatkezelést. A program csomagszerkezete a 2. ábrán látható.
- A program szerkezetét két projektre osztjuk implementációs megfontolásból: a **Persistence** és **Model** csomagok a program felületfüggetlen projektjében, míg a **ViewModel** és **View** csomagok a WPF függő projektjében kapnak helyet.

**2. ábra: Az alkalmazás csomagdiagramja**

Perzisztencia (3.ábra):

- Az adatkezelés feladata a **Malom** táblával kapcsolatos információk tárolása, valamint a betöltés/mentés biztosítása.
- A hosszú távú adattárolás lehetőségeit az **IGamePersistence** interfész adja meg, amely lehetőséget ad az állapot mentésére (**SaveGame**) és betöltésére (**LoadGame**).

- Az interfészt szöveges fájl alapú adatkezelésre a **GamePersistence** osztály valósítja meg. A fájlkezelés során fellépő hibákat a **GameDataException** kivétel jelzi.
- A program az adatokat szöveges fájlként tudja eltárolni, melyek a **txt** kiterjesztést kapják. Ezeket az adatokat a programban bármikor be lehet tölteni, illetve ki lehet menteni az aktuális állást.
- A fájl első sora megadja a jelenlegi játékost, fázist és leszedési módot. A második sor a számlálót (bábuk száma, stb.). A harmadik sor a tábla állapotát (24 érték szóközzel, enumként int-re konvertálva).

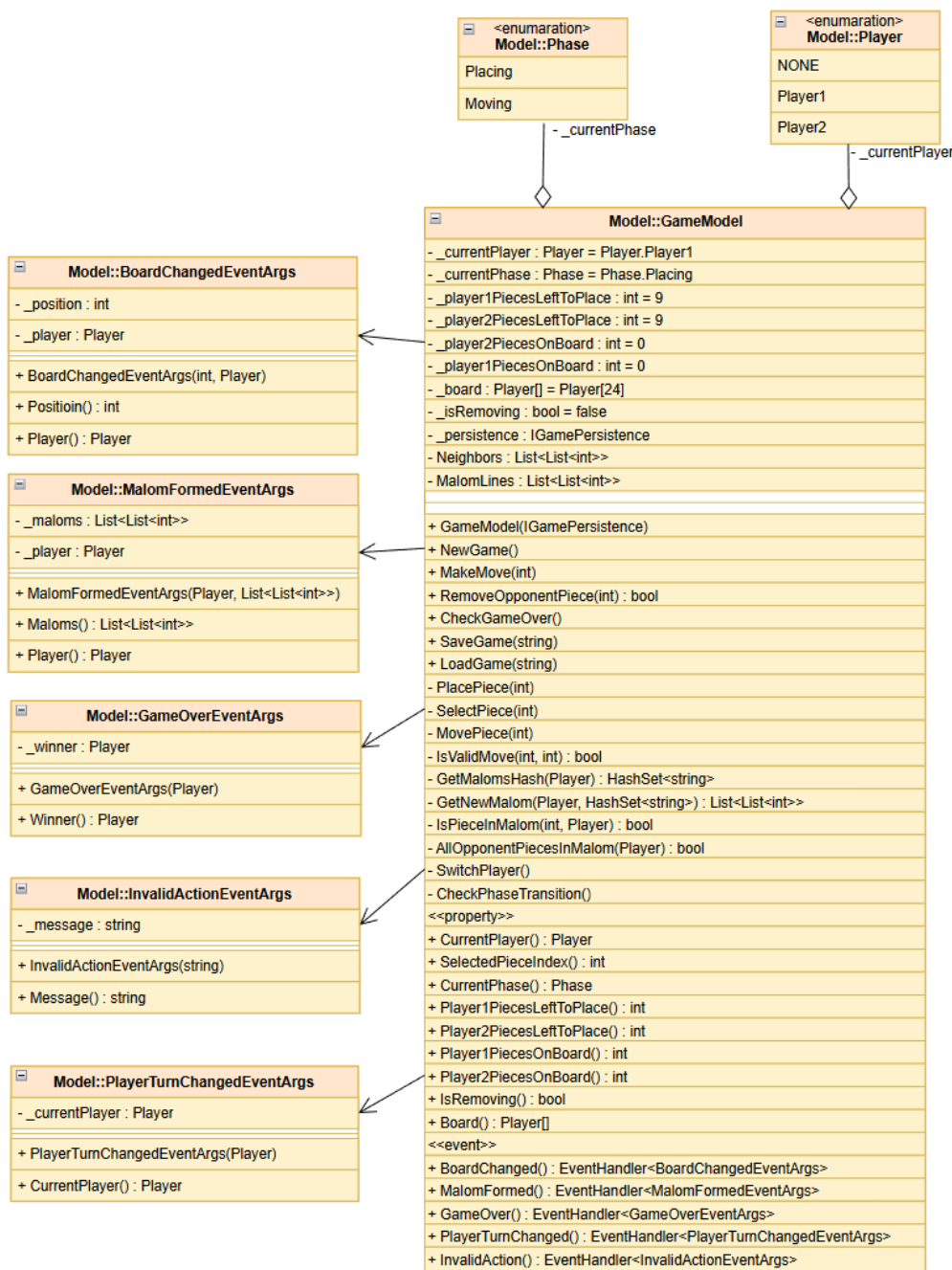


3. ábra: A Persistence csomag osztálydiagramja

Modell (4.ábra):

- A modell lényegi részét a **GameModel** osztály valósítja meg, amely szabályozza a tábla tevékenységeit, valamint a játék egyéb paramétereit, úgymint a bábuk száma és a fázis (**_currentPhase**). A típus lehetőséget ad új játék kezdésére (**NewGame**), lépésre (**MakeMove**) és ellenfél bábujának eltávolítására (**RemoveOpponentPiece**). Új játéknál a tábla üres.
- A modell példányosításkor megkapja az adatkezelés felületét, amelynek segítségével lehetőséget ad betöltésre (**LoadGame**) és mentésre (**SaveGame**).

- A tábla állapotváltozásáról a **BoardChanged** esemény tájékoztat. Az esemény argumentuma (**BoardChangedEventArgs**) tárolja a megváltozott mező pozícióját és a játékos.
- A malom kialakulásáról a **MalomFormed** esemény, míg a játék végéről a **GameOver** esemény tájékoztat. Az események argumentumai (**MalomFormedEventArgs**, **GameOverEventArgs**) tárolják a játékos, malmokat vagy győztest.
- A játékosváltásról a **PlayerTurnChanged**, érvénytelen lépésről az **InvalidAction** esemény tájékoztat.
- Nézet:
- A nézetet a **GameForm** osztály biztosítja, amely tárolja a modell egy példányát (**_model**), valamint az adatkezelés konkrét példányát (**_persistence**).
- A nézetet a **GameForm** osztály biztosítja, amely tárolja a modell egy példányát (**_model**), valamint az adatkezelés konkrét példányát (**_persistence**).
- A program teljes statikus szerkezete a 3. ábrán látható.

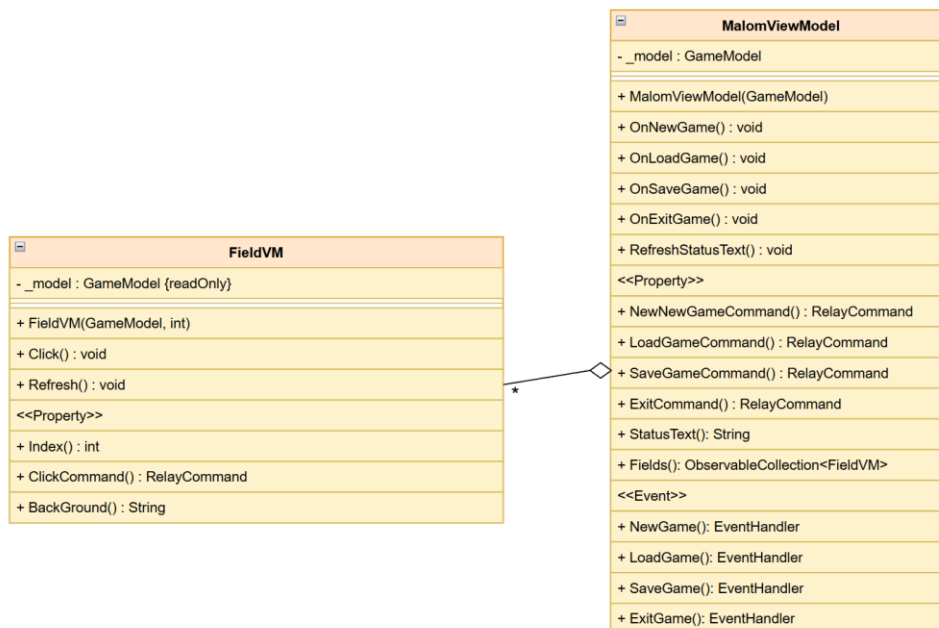


3. ábra: Az alkalmazás osztálydiagramja

Nézetmodell (5.ábra):

- A nézetmodell megvalósításához felhasználjuk a **CommunityToolkit.Mvvm** csomagból elérhető **RelayCommand** és **ObservableObject** osztályokat.
- A nézetmodell feladatait a **MalomViewModel** osztály látja el, amely parancsokat biztosít az új játék kezdéséhez, játék betöltéséhez, mentéséhez, valamint a kilépéshez. A parancsokhoz eseményeket kötünk, amelyek a parancs lefutását jelzik a vezérlőnek. A nézetmodell tárolja a modell egy hivatkozását (**_model**), de csupán információkat kér le tőle, illetve a játéknehézséget szabályozza. Direkt nem avatkozik a játék futtatásába.

- A játékműködés számára egy külön mezőt biztosítunk (**FieldVM**), amely eltárolja a pozíciót, háttérszínt, valamint a kattintás parancsát. A mezőket egy felügyelt gyűjteménybe helyezzük a nézetmodellbe (**Fields**).



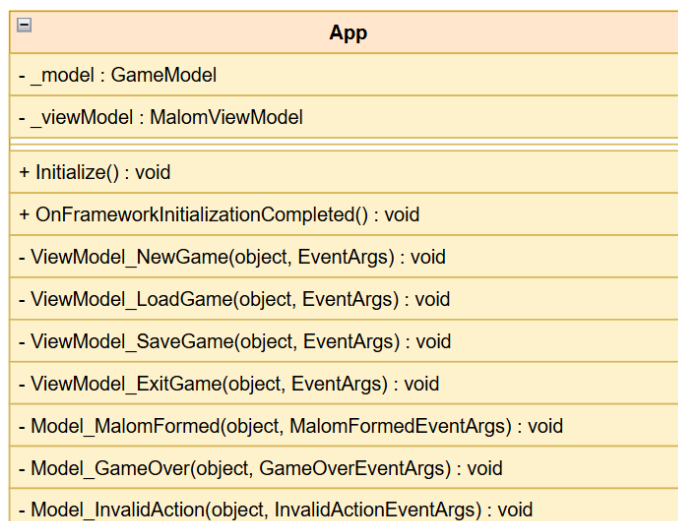
5. ábra: A nézetmodell osztálydiagramja

Nézet:

- A nézetet ablakok, egyedi vezérlők és dialógusablakok használatával valósítjuk meg.
- Betöltéshez és mentéshez a **StorageProvider** osztály által biztosított **OpenFilePickerAsync** és **SaveFilePickerAsync** metódusokat használjuk, amelyek platformspecifikus dialógusablakokat nyitnak meg.
- Felugró üzenetek megjelenítéséhez a **MessageBox.Avalonia** NuGet csomagot használjuk.

Környezet (6.ábra):

- Az **App** osztály feladata az alkalmazás vezérlése, a rétegek példányosítása és az események feldolgozása.
- A **OnFrameworkInitializationCompleted** metódus felüldefiniálásával kezeljük a nézet platform specifikus megjelenítését, továbbá az alkalmazás életciklusát a megfelelő eseményekre történő feliratkozással.



6. ábra: A vezérlés osztálydiagramja

Tesztelés:

- A modell funkcionalitása egységtesztek segítségével lett ellenőrizve a **GameModelTest** osztályban.
- A modell perzisztenciáját egy **IGamePersistence** interfészből származtattuk le, így azt a teszt projektben mockolhatjuk Moq segítségével.
- Az alábbi tesztesetek kerültek megvalósításra:
 - **NewGameInitializationTest**: Új játék indítása, a tábla üres, játékosok állapota és események ellenőrzése.
 - **PlacingPhaseTest**: Helyezési fázis lépéseinek ellenőrzése, beleértve érvényes helyezést.
 - **InvalidPlacingPhaseTest**: Érvénytelen lépések a lerakó fázisban
 - **MalomFormationTest**: Malom kialakulása, leszedés hatásai.
 - **RemovalTest**: Bábu eltávolításának tesztje malom után
 - **InvalidRemovalTests**: Érvénytelen eltávolítási kísérletek
 - **MovingPhaseTest**: Mozgatási fázis lépései, érvényes/érvénytelen mozgatások.
 - **GameOverConditionsTest**: Játék vége ellenőrzése, ha bábuk száma 3 alá csökken.
 - **LoadSaveTest**: A játék modell mentésének és betöltésének tesztelése mockolt perzisztencia réteggel.